

Task 1

1.1 Project Description

York university students interact with eClass across multiple courses every term. Each course contains messy categorization of information like announcements, forum posts, assignment descriptions, quizzes, lecture slides, and links. Students have to manually read through dozens of announcements and reorient themselves to traverse through each course. This becomes time consuming and prone for students to interpret things incorrectly. As a result, students miss deadlines, priorities, and waste time looking for information.

The targeted users are YorkU students enrolled in multiple eClass courses per term. Also, students with accessibility needs who benefit from simple concise summaries, priorities ranking, language interaction, etc.

The AI agent will take in course activity with a live eClass fetch where it grabs info from the site, it summarizes announcements and forums, ranks deadlines and assessments by priority, detects what changed, generates study plans, answers questions about the course. All features are reachable from both GUI and CLI.

The AI layer will be using Qwen3.6 via cloud API and qwen3.5 with Ollama for local/offline runs, with embeddings for RAG over slides and PDFs.

The LLM is gonna be a multi-agent system that interacts with a system that drives the LLM and uses a set of tools like eClass fetcher, PDF parser, etc. There is also a background component that watches for new items between each session.

1.2 Feature Specification

F01 - Course announcement Summarizer (AI-based)

- The user selects courses in GUI and clicks “Summarize Announcements”. The agent retrieves new announcements, analyzes them, and generates a concise summary. The summary is displayed in the GUI and can be expanded or regenerated.

F02 - Deadline & Assessment Summarizer (Hybrid)

- The user opens the “Deadlines” tab and selects a course. The agent reads through course pages, syllabus, and announcements, it then extracts deadlines, weights, and formats, and produces a list. The list is displayed in the GUI and can be edited by the user to correct any errors.

F03 - eClass change detector (Hybrid)

- The user can toggle a “Watch” button on a course. It will then periodically check eClass content when the program is open, or update the eClass content when the program is opened again. The log of new content is displayed in the GUI.

F04 - Task priority ranker (AI-based)

- The user can sort the deadlines tab and rank them with a score. The deadlines tab is then sorted with the new ranked list.

F05 - Progress Tracker (Hybrid)

- The user marks tasks as complete via checkboxes in the GUI. The agent updates progress scores and re prioritises remaining tasks.

F06 - Final grade calculator (Deterministic)

- The user opens the “Grades” tab and enters scores per assessment. The agent computes the total grade and output current average. The user can also have a grade goal where the agent can highlight grades needed per assessment to reach a certain final grade.

F07 - Language Q&A (AI-based)

- The user can type questions in the chat panel. The agent embeds the questions and retrieves relevant chunks from stored course materials, and generates a answer. The answer is displayed in the GUI.

F08 - Study plan generator (AI-based)

- The user can set their availability in the “Study” tab and generate a plan. The agent turns the tasks list into subtasks and estimates the time needed to complete. The plan is displayed in the GUI as a schedule and can be adjusted.

F09 - Slides Summary / Notes (AI-based)

- The user can upload or sync lecture slides and click “notes”. The agent reads the file, extracts key information, and generates structured study notes in a certain style. The notes are displayed in the GUI.

F10 - Weekly eClass report (AI-based)

- The user can click “generate report”. The agent gathers outputs from announcements, deadlines, grades, and puts them in a report. The report is displayed in the GUI and can be exported as a PDF.

F11 - Practice quiz generator (AI-based)

- The user opens the “practice” tab, selects a course, topic, and lecture notes. The agent retrieves relevant materials, generates practice questions, and grades the user’s answers. The quiz and results are displayed in the GUI.

Task 2

Task 2: Design Your System Using UML

Use UML diagrams to make the architecture and behavior of your proposed system clear.

You must provide:

1. Class Diagram
2. Use-Case Diagram
3. Sequence Diagrams

The diagrams must be consistent with each other and with the features described in Task 1.

2.1 Class Diagram

Create a UML class diagram representing the major structure of your software.

The diagram should include:

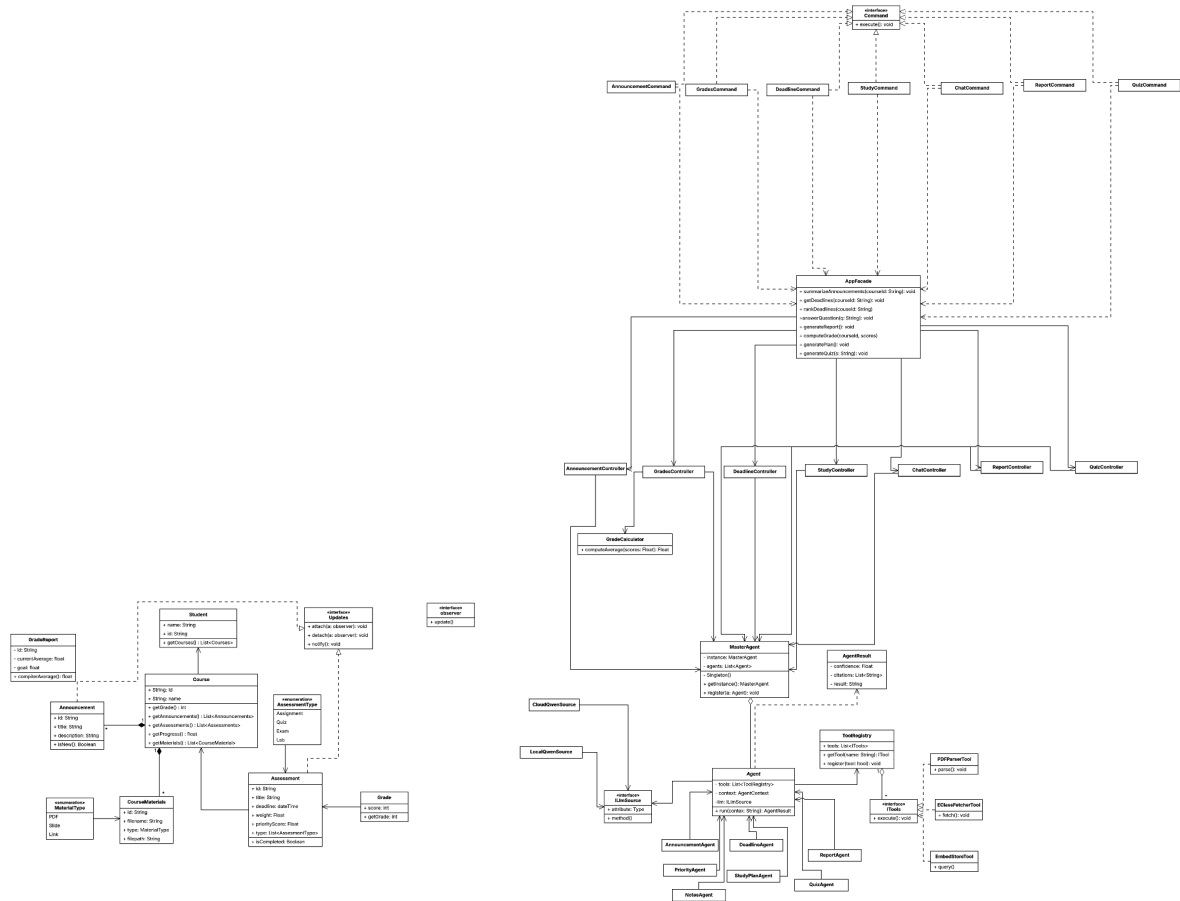
- major classes and interfaces;
- important attributes;
- important methods;
- associations;
- dependencies;
- inheritance;
- aggregation/composition, where appropriate;
- multiplicities were meaningful.

Your class diagram should show both the traditional software components and the AI/agent components.

Design Patterns

Your design must demonstrate the meaningful application of at least five design patterns covered in the course. Possible patterns include:

- Strategy
- Observer
- Factory Method
- Abstract Factory
- Singleton
- Adapter
- Facade
- Command
- State
- Template Method
- Decorator
- Composite
- Iterator
- MVC



Do not add patterns simply to satisfy the numerical requirement. Each pattern should solve an identifiable design problem in your application. For every pattern used, explain:

- What design problem does the pattern address?
- Which classes participate in the pattern?
- What role does each class play?
- Why is this pattern appropriate?
- What would be more difficult if the pattern were not used?

Pattern 1 - Template Method

All nine agents follow the same skeleton pattern

1. Validate
2. Fetch data with tools
3. Build prompt
4. Call LLM
5. Parse into AgentResult

The Agent plays the AbstractClass role, which defines the template method with steps. Each function agent plays the ConcreteClass role. The template method is appropriate because it lets me write the flow once without creating multiple classes and makes it easier to edit those nine classes.

Pattern 2 - Strategy

The system must call an LLM through one of two changeable backends, Qwen3.6 via cloud API or qwen3.5 via Ollama. This is chosen at runtime based on user preference or network availability. The `ILlmSource` plays the Strategy role, `CloudQwenSource` and `LocalQwenSource` are `ConcreteStrategies`, and `Agent` are the contexts that hold a `ILlmSource` reference. Strategy is appropriate because the 2 LLM sources do the same job differently, and having one interface makes the fallback easier if one fails. Without it, every agent would need to check if the LLM is online.

Pattern 3 - Observer

When new eClass content arrives or a task is marked done, many things need to happen like re ranking priority, progress view, and updating GUI. The `Updates` and `observer` are the interfaces and `courses` and `TaskList` are subjects that call `notify()` when a change happens. This pattern fits because the same event changes many parts of the app, which makes the list of listeners to grow and inefficient. Without it, domain classes would have to import UI classes to add a new listener and edit the domain.

Pattern 4 - Command

All features have to work from both GUI and CLI. `Command` is the interface with `execute()`, each feature gets its own command and `CommandInvoker` stores the history and runs commands. This pattern fits because one command per feature makes GUI and CLI share the same calls, and the invoker becomes the only place for input handling. Without it, GUI and CLI would need to duplicate many logics.

Pattern 5 - Facade

`AppFacade` has one method per feature and hides all of it. The clients only call `AppFacade`. This pattern fits because every feature has the same actions and it keeps the UI from being intertwined with the big system. Without it the UI would need to call the AI agent system, any internal changes in the system would make the UI and GUI behave differently.

2.2 Use-Case Diagram and Use-Case Descriptions

Create a UML use-case diagram that represents the interactions between the system and its external actors. Your use cases should collectively cover all major features of the project.

Identify all relevant actors, such as:

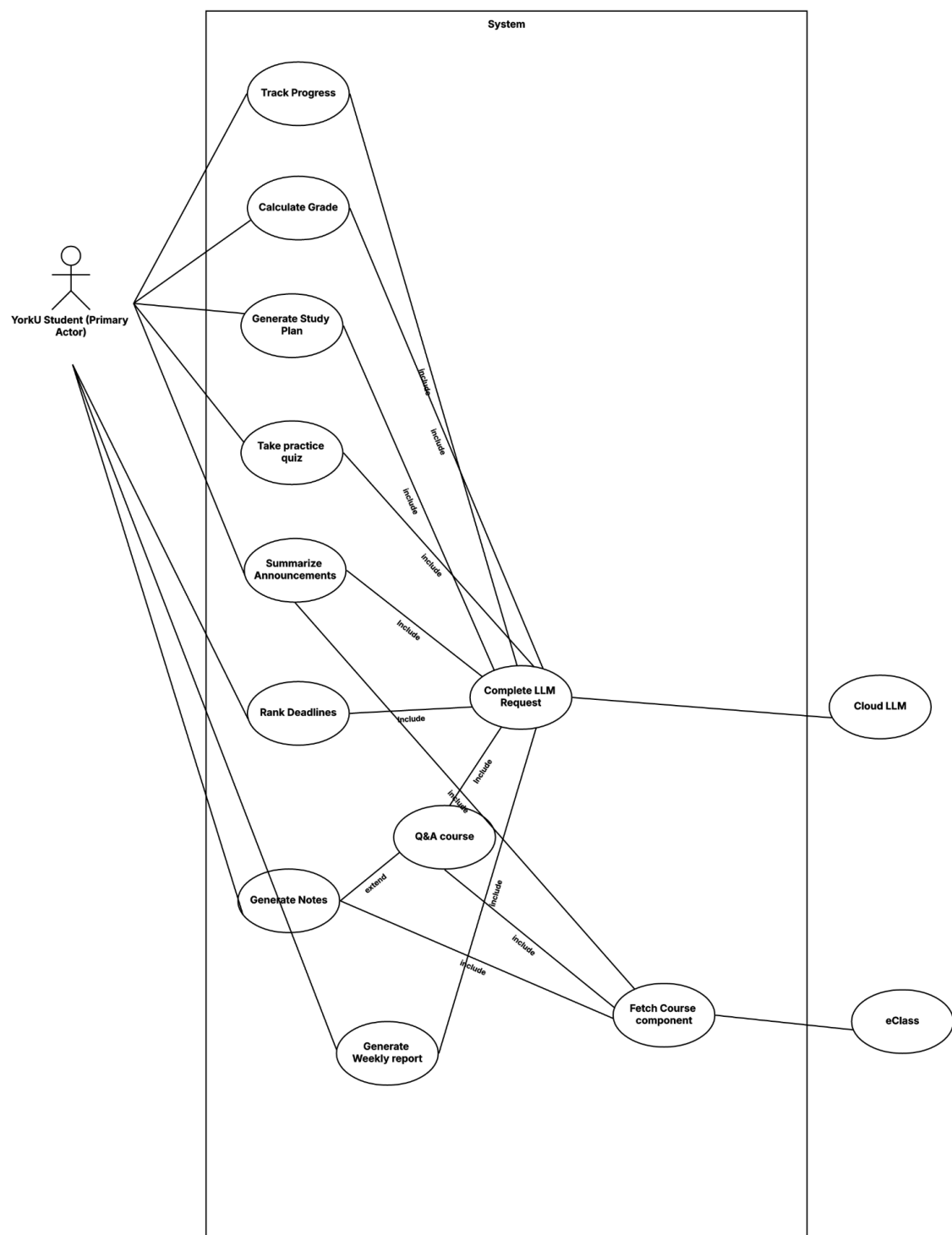
- primary user;
- administrator, if applicable;
- external services;
- external APIs/tools;
- AI services, where appropriate.

For each major use case, provide a description containing:

- Use Case ID;
- Use Case Name;
- Actor(s);
- Goal;
- Preconditions;

- Trigger;
- Main Success Scenario;
- Alternative/Exception Flows;
- Postconditions;
- Related Feature(s).

The use-case diagram and descriptions should make it possible to understand how users interact with all major functionality of your system.



2.3 Sequence Diagrams

Create sequence diagrams to show how objects collaborate at runtime to implement the major features/use cases. Your sequence diagrams should show:

- the initiating actor;
- GUI/boundary objects;
- controller objects;
- domain objects;

- AI/agent components;
- external tools/services where applicable;
- important method calls;
- returned results;
- alternative/error flows where appropriate.

Your sequence diagrams should use the actual classes and important methods defined in your class diagram.

You do not necessarily need one sequence diagram for every individual feature if several closely related features share the same interaction structure. However, all important system behaviors must be represented by the submitted sequence diagrams

Task 3

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Course announcement summarizer (AI-based)	AI	UC01 Summarize Announcements	SummarizeAnnouncementsCommand, AppFacade, AnnouncementController, MasterAgent, AnnouncementAgent, EclassFetcherTool, ILLmSource			Command, Facade, Strategy, Factory
F02	Deadline & assessment summarizer (Hybrid)	Hybrid	UC02 View & Rank Deadlines	DeadlineCommand, AppFacade, DeadlineController, MasterAgent, DeadlineAgent, EclassFetcherTool, PDFParserTool, ILLmSource			Command, Facade, Strategy, Factory
F03	eClass change detector (Hybrid)	Hybrid		DeadlineController, MasterAgent, DeadlineAgent, EclassFetcherTool			Observer, Facade
F04	Task priority ranker (AI-based)	AI	UC02 View & Rank Deadlines	DeadlineCommand, AppFacade, DeadlineController, MasterAgent, PriorityAgent, ILLmSource, Deadline			Command, Facade, Strategy
F05	Progress tracker (Hybrid)	Hybrid	UC04 Track Progress	DeadlineCommand, AppFacade, DeadlineController, Deadline, Course, Observer			Command, Facade, Observer
F06	Final grade calculator (Deterministic)	Deterministic	UC05 Calculate Final Grade	GradesCommand, AppFacade, GradesController, GradeCalculator, Grade, GradeReport, Assessment			Command, Facade
F07	Language Q&A (AI-based, RAG)	AI	UC06 Ask Course Q&A	ChatCommand, AppFacade, ChatController, MasterAgent, ChatAgent,			Command, Facade, Strategy, Factory

				EmbeddingTool, ILlmSource, CourseMaterial			
F08	Study plan generator (AI-based)	AI	UC07 Generate Study Plan	StudyCommand, AppFacade, StudyController, MasterAgent, StudyPlanAgent, ILlmSource, Deadline, Assessment			Command, Facade, Strategy
F09	Slides summary / notes (AI-based)	AI	UC08 Generate Notes from Slides	StudyCommand, AppFacade, StudyController, MasterAgent, NotesAgent, PDFParserTool, EmbeddingTool, ILlmSource, CourseMaterial			Command, Facade, Strategy, Factory
F10	Weekly eClass report (AI-based)	AI	UC09 Generate Weekly Report	ReportCommand, AppFacade, ReportController, MasterAgent, ReportAgent, AnnouncementAgent, DeadlineAgent, GradesController, ILlmSource			Command, Facade, Strategy, Factory
F11	Practice quiz generator (AI-based)	AI	UC10 Take Practice Quiz	QuizCommand, AppFacade, QuizController, MasterAgent, QuizAgent, EmbeddingTool, ILlmSource, CourseMaterial			Command, Facade, Strategy, Factory